

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО
ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федерального государственного бюджетного
образовательного учреждения высшего образования
**«РОССИЙСКИЙ ЭКОНОМИЧЕСКИЙ УНИВЕРСИТЕТ ИМЕНИ
Г.В.ПЛЕХАНОВА»**

Техникум Пермского института (филиала)

Практическая №1
по дисциплине «Поддержка и тестирование программных
модулей»

по теме: «Разработка системы управления складом»

Исполнитель:
Пищальникова Вика ИПо-32

Пермь, 2026

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ	4
1. АНАЛИЗ ЗАДАНИЯ И ПЛАНИРОВАНИЕ СТРУКТУРЫ ПРОЕКТА.....	5
2. РАЗРАБОТКА МОДЕЛЕЙ ДАННЫХ.....	6
2.1. Выбор подхода к идентификации	6
2.2. Модель организации.....	6
2.3. Модель склада.....	6
2.4. Модель товара.....	7
2.5. Модель накладной.....	7
2.6. Справочные модели.....	8
3. КОНТЕКСТ ХРАНЕНИЯ ДАННЫХ.....	9
4. РЕАЛИЗАЦИЯ СЕРВИСОВ.....	10
4.1. OrgService - управление организациями	10
4.2. WarehouseService - управление складами.....	10
4.3. InvoiceService - управление накладными и согласование	10
4.4. ProductService - управление товарами в накладной	11
5. РАЗРАБОТКА ПОЛЬЗОВАТЕЛЬСКОГО ИНТЕРФЕЙСА.....	12
5.1. Общая структура интерфейса.....	12
5.2. Модуль управления организациями	12
5.3. Модуль управления складами	13
5.4. Модуль управления товарами	13
5.5. Модуль управления накладными.....	14
6. НАВИГАЦИЯ МЕЖДУ МОДУЛЯМИ И УПРАВЛЕНИЕ КОНТЕКСТОМ.....	15
7. ВАЛИДАЦИЯ ДАННЫХ И ОБРАБОТКА ОШИБОК	16
8. ОРГАНИЗАЦИЯ КОДА И СТРУКТУРА РЕШЕНИЯ.....	17
ЗАКЛЮЧЕНИЕ.....	18

ВВЕДЕНИЕ

Данная сопроводительная записка описывает процесс разработки настольного приложения «Управление складом», выполненного в рамках практической работы по дисциплине «Поддержка и тестирование программных модулей». В записке изложен ход работы, принятые архитектурные и технические решения, а также обоснование выбора тех или иных подходов на каждом этапе.

Целью работы являлось создание функционального WPF-приложения, реализующего учёт справочных данных организации: юридических лиц, складов и товарной номенклатуры. Приложение должно обеспечивать полный набор операций CRUD для каждой из сущностей, а также поддерживать управление накладными — входящими и исходящими — с изменением остатков товаров на складах при их согласовании.

Технологический стек был задан условием задания: язык C#, фреймворк WPF, архитектура MVVM. Хранение данных реализовано не через базу данных, а через коллекции в памяти с предзаполнением тестовыми данными. Вся бизнес-логика и модели вынесены в отдельную библиотеку классов, что обеспечивает чистое разделение ответственности между слоями приложения.

При разработке я стремилась выстроить логичную и расширяемую структуру проекта, где каждый компонент выполняет строго одну задачу. Ниже описан каждый этап работы с точки зрения принятых решений и хода мышления в процессе реализации.

1. АНАЛИЗ ЗАДАНИЯ И ПЛАНИРОВАНИЕ СТРУКТУРЫ ПРОЕКТА

Прежде чем приступить к написанию кода, я выделила ключевые сущности, с которыми предстоит работать: организации, склады, товары, накладные и строки накладных.

Организация является корневой сущностью: к ней привязываются склады, к складам - товары и накладные. Это иерархическая структура, и работать с ней нужно именно сверху вниз: сначала выбирается организация, затем в её контексте отображаются склады, затем в контексте склада - товары и накладные.

В задании было сказано использовать библиотеку классов для хранения моделей и сервисов.

В итоге я пришла к следующей структуре: два проекта внутри одного решения — библиотека классов WarehouseData, содержащая все модели, контекст хранения данных и сервисы, и WPF-приложение Warehouse, отвечающее исключительно за отображение и взаимодействие с пользователем. Такое разделение позволяет менять интерфейс, не затрагивая бизнес-логику, и наоборот.

2. РАЗРАБОТКА МОДЕЛЕЙ ДАННЫХ

2.1. Выбор подхода к идентификации

Поскольку данные хранятся в коллекциях, а не в базе данных с автоинкрементными ключами, нужно было реализовать генерацию идентификаторов вручную.

Я выбрала подход со статическим счётчиком в каждом классе. Каждый раз, когда создаётся новый объект Organization, Warehouse, Invoice или Product, счётчик увеличивается на единицу, и это значение присваивается полю идентификатора. Для хранения идентификаторов использован тип BigInteger из пространства имён System.Numerics - он позволяет хранить большое целое число, что исключает риск переполнения при большом количестве создаваемых объектов.

2.2. Модель организации

Класс Organization является точкой входа в иерархию данных. Он содержит идентификатор, наименование и коллекцию складов типа ObservableCollection<Warehouse>. Использование ObservableCollection принципиально важно для WPF: эта коллекция автоматически уведомляет интерфейс об изменениях - добавлении и удалении элементов, что избавляет от необходимости вручную обновлять списки на экране.

Конструктор класса принимает наименование и сразу присваивает идентификатор через статический счётчик.

2.3. Модель склада

Класс Warehouse хранит идентификатор, наименование, адрес, идентификатор родительской организации, а также две коллекции: товары

(products) и накладные (Invoices). Обе коллекции - ObservableCollection, по той же причине, что и у организации.

2.4. Модель товара

Класс Product помимо идентификатора и наименования, содержит артикул, единицу измерения, цену, остаток на складе, описание, путь к фотографии, а также ссылки на связанные объекты: категорию, производителя и поставщика.

Связанные объекты хранятся двумя способами: как ссылка на объект (например, Category) и как числовой идентификатор (CategoryId). Ссылка нужна для удобного отображения в интерфейсе, идентификатор - для логики сравнения и поиска.

Поле StockQuantity отвечает за текущий остаток товара на складе. Именно это поле изменяется при согласовании накладных - увеличивается для входящих и уменьшается для исходящих. При достижении нулевого остатка товар автоматически удаляется с остатков склада.

2.5. Модель накладной

Класс Invoice описывает документ перемещения товаров. Ключевые поля - номер накладной, дата, тип (входящая или исходящая) и статус (не согласована или согласована). Список товаров в накладной хранится как ObservableCollection<Product>.

Типы и статусы реализованы через перечисления InvType и InvStatus с русскоязычными значениями.

Статус «Согласована» является финальным: согласованную накладную нельзя ни изменить, ни удалить.

2.6. Справочные модели

Помимо основных сущностей, в библиотеке реализованы три справочных класса: `Category` (категория товара), `Manufacturer` (производитель) и `Supplier` (поставщик). Каждый из них содержит идентификатор и наименование. Справочники хранятся в контексте приложения как обычные списки `List<T>` и используются при заполнении выпадающих списков в формах редактирования товаров.

3. КОНТЕКСТ ХРАНЕНИЯ ДАННЫХ

Класс `ApplicationContext` выполняет роль, аналогичную `DbContext` в проектах с `Entity Framework` - он является единым источником данных для всего приложения.

Центральным полем контекста является `ObservableCollection<Organization> orgs` - коллекция организаций, которая содержит всю иерархию данных: внутри каждой организации - склады, внутри каждого склада - товары, накладные и строки накладных. Таким образом, имея ссылку на контекст, можно получить доступ к любым данным приложения.

При создании контекста автоматически вызывается метод `OrgsFill()`, который заполняет коллекции тестовыми данными. Это сделано для того, чтобы при каждом запуске приложения была готовая демонстрационная среда без необходимости вводить данные вручную.

Тестовые данные охватывают весь спектр сущностей: две организации («Рога и копыта, ООО» и «Пупкин и сыновья, ООО»), по два склада для каждой, набор товаров на каждом складе из разных категорий, а также по две тестовых накладные - входящие и исходящие - с товарами.

Справочные данные - категории, производители, поставщики - также инициализируются в `OrgsFill()` и хранятся в соответствующих списках контекста. При создании или редактировании товара форма обращается к этим спискам для заполнения выпадающих меню.

4. РЕАЛИЗАЦИЯ СЕРВИСОВ

Сервисный слой - это сердце бизнес-логики приложения. Каждый сервис отвечает за операции над одним типом сущностей и работает через контекст приложения. В проекте реализованы четыре сервиса: OrgService, WarehouseService, ProductService и InvoiceService.

Все сервисы реализуют соответствующие интерфейсы (IOrgService, IWarehouseService и так далее).

4.1. OrgService - управление организациями

Сервис организаций реализует методы Add, Update, Delete и GetContext. При добавлении проверяется, что переданный объект не равен null, его наименование не пустое, и такого объекта ещё нет в коллекции. При удалении проверяется наличие объекта в коллекции. При обновлении - наличие организации с соответствующим идентификатором.

Метод GetContext возвращает ссылку на контекст приложения.

4.2. WarehouseService - управление складами

Сервис складов принимает в конструктор не только контекст, но и конкретную организацию - владельца складов. Это сделано намеренно: сервис работает только с теми складами, которые принадлежат выбранной организации, что соответствует иерархической модели данных.

Логика методов аналогична OrgService: при добавлении проверяется наличие склада в коллекции организации, при удалении - принадлежность склада текущей организации. Обновление находит склад по идентификатору и заменяет объект в коллекции целиком.

4.3. InvoiceService - управление накладными и согласование

При вызове Update выполняется несколько проверок: накладная не должна быть null, её статус не должен быть «Согласована», и список товаров не должен быть пустым.

Для входящих накладных алгоритм перебирает товары и для каждого ищет совпадение по артикулу на складе. Если товар уже есть — увеличивает его остаток. Если нет - добавляет товар на склад.

Для исходящих накладных алгоритм сначала проверяет, что все позиции в накладной есть на складе и их достаточно для списания. Только если все проверки прошли — выполняется фактическое списание. При достижении нуля товар удаляется из остатков.

После успешного согласования статус накладной устанавливается в «Согласована». Согласованную накладную нельзя удалить - это тоже проверяется в методе Delete.

4.4. ProductService - управление товарами в накладной

Сервис товаров работает не с остатками склада напрямую, а со списком товаров конкретной накладной. Это позволяет формировать состав накладной перед её согласованием: добавлять позиции, редактировать количество и цену, удалять ненужные строки. При добавлении проверяется отсутствие дубликата объекта в списке накладной.

5. РАЗРАБОТКА ПОЛЬЗОВАТЕЛЬСКОГО ИНТЕРФЕЙСА

5.1. Общая структура интерфейса

Пользовательский интерфейс построен на WPF с применением архитектурного шаблона MVVM. Главное окно (MainWindow) является контейнером, в котором последовательно отображаются экраны в зависимости от выбранного контекста. Навигация реализована через смену содержимого контейнера.

Интерфейс следует иерархии данных: первым открывается список организаций, выбор организации открывает список её складов, выбор склада - список товаров и накладных.

Для единообразного оформления всех окон используется словарь ресурсов DictionaryBasic.xaml, подключённый на уровне приложения в App.xaml. В нём описаны общие стили для кнопок, списков, текстовых полей и других элементов управления. Это позволяет изменить внешний вид всего приложения, отредактировав один файл.

5.2. Модуль управления организациями

Экран организаций (OrgsView) содержит ListView со списком всех организаций и кнопки управления. При правом клике на элемент списка открывается контекстное меню с вариантами: новая организация, изменить, удалить. Кнопки «ОК» и «Закрыть» управляют подтверждением выбора.

Добавление и редактирование организации реализованы через диалоговое окно OrgModWindow, которое запрашивает только одно поле - наименование компании.

Удаление организации реализовано с защитой: пользователю предлагается ввести наименование организации для подтверждения.

Выбранная организация сохраняется как текущий контекст и передаётся в следующий уровень иерархии - модуль складов. Статус-строка внизу экрана всегда показывает, какая организация выбрана в данный момент.

5.3. Модуль управления складами

Экран складов отображается внутри основного окна при выборе организации. В заголовке видно, к какой организации относятся отображаемые склады. Список складов также оформлен как ListView.

Управление складами доступно через контекстное меню: новый склад, изменить склад, удалить склад. Диалог WhModWindow запрашивает наименование и адрес склада. Адрес добавлен в модель, чтобы обеспечить достаточную информативность справочника складов.

Дополнительно в меню склада есть пункт «Импорт товаров» - он открывает диалог загрузки товаров из CSV-файла. Формат файла: каждая строка содержит артикул, наименование, единицу измерения, цену и количество, разделённые точкой с запятой. Это позволяет быстро наполнить склад товарами из внешнего источника, не вводя каждую позицию вручную.

Выбранный склад, как и организация, сохраняется как контекст и передаётся в модули товаров и накладных.

5.4. Модуль управления товарами

Товары отображаются в виде детализированной таблицы (DataGrid) с колонками: фото, наименование, категория, производитель, поставщик, цена, остаток, скидка.

Над таблицей расположено поле поиска, которое фильтрует список товаров по наименованию в реальном времени.

Форма добавления и редактирования товара (ProdModWindow) содержит все поля модели Product. Выпадающие списки для категории, производителя и поставщика заполняются из справочников контекста. Поле фотографии позволяет выбрать файл изображения через стандартный диалог открытия файла.

В строке контекста главного окна при работе с товарами отображается и организация, и склад.

5.5. Модуль управления накладными

Накладные отображаются в отдельном пользовательском элементе управления (UserControlInvoices), который встраивается в главное окно. Список накладных выводит номер, дату, тип и статус каждой. При выборе накладной в нижней части отображается таблица товаров, входящих в неё.

Создание накладной (InvModWindow) требует указания номера и типа. После создания в накладную можно добавлять товары, редактировать их количество и цену, а также удалять строки.

Кнопка «Согласовать» запускает метод Update сервиса InvoiceService. После успешного согласования статус накладной меняется на «Согласована», кнопки редактирования и удаления деактивируются, а остатки товаров на складе немедленно обновляются и видны в таблице модуля товаров.

6. НАВИГАЦИЯ МЕЖДУ МОДУЛЯМИ И УПРАВЛЕНИЕ КОНТЕКСТОМ

Передача ссылок через конструкторы: каждый следующий уровень иерархии получает ссылки на контекст приложения, выбранную организацию и (там, где нужно) выбранный склад.

Статус-строка главного окна постоянно отображает текущий контекст: выбранную организацию и склад. Это позволяет пользователю в любой момент понимать, с данными какого склада и какой организации он работает. При смене организации или склада статус-строка обновляется.

Меню главного окна организовано по разделам: Файл, Склады, Товары, Накладные, Справочники. Пункты меню активируются или деактивируются в зависимости от того, выбрана ли организация и склад.

7. ВАЛИДАЦИЯ ДАННЫХ И ОБРАБОТКА ОШИБОК

Валидация данных реализована на двух уровнях. На уровне сервисов проверяется корректность объектов перед изменением коллекций - метод возвращает false, если переданные данные некорректны. На уровне интерфейса проверка выполняется перед отправкой данных в сервис, и при обнаружении ошибки пользователю показывается сообщение.

Обязательные поля - наименование организации, наименование склада, артикул и наименование товара, номер накладной - проверяются на пустоту и пробелы. Числовые поля (цена, остаток) проверяются на корректность типа и диапазон: цена не может быть отрицательной, остаток должен быть целым неотрицательным числом.

Операция удаления организации защищена дополнительным подтверждением: пользователь должен ввести название организации, которую хочет удалить. Это защита от случайного удаления важных данных. Та же логика применяется и к удалению складов.

При попытке согласовать накладную с пустым списком товаров или исходящую накладную, в которой количество превышает остаток на складе, пользователь получает информативное сообщение об ошибке с указанием причины отказа.

8. ОРГАНИЗАЦИЯ КОДА И СТРУКТУРА РЕШЕНИЯ

Решение состоит из двух проектов: WarehouseData (библиотека классов) и Warehouse (WPF-приложение). Такое разделение обеспечивает независимость бизнес-логики от интерфейса.

В библиотеке WarehouseData выделены следующие папки: Models (модели сущностей), Context (класс ApplicationContext), Services (сервисы с интерфейсами).

В WPF-приложении Warehouse структура организована по типу элементов: папка Views содержит окна и страницы, папка Controls - пользовательские элементы управления, папка Resources - словарь стилей. Файл WindowCommands.cs содержит определения команд WPF, используемых в меню и кнопках.

ЗАКЛЮЧЕНИЕ

В ходе выполнения практической работы было разработано настольное WPF-приложение «Управление складом», реализующее трёхуровневую иерархию данных: организации - склады - товары с накладными. Приложение обеспечивает полный набор операций CRUD для каждой сущности, а также бизнес-логику согласования накладных с автоматическим изменением остатков.

Архитектурное разделение на библиотеку классов и WPF-приложение обеспечило чистое разграничение ответственности: бизнес-логика не зависит от интерфейса, интерфейс не содержит бизнес-логики. Использование интерфейсов для сервисов делает систему расширяемой.

Работа над проектом позволила на практике освоить проектирование многоуровневой иерархии данных в памяти, реализацию бизнес-логики в сервисном слое, построение WPF-интерфейса с контекстной навигацией и применение паттерна MVVM в реальном проекте.

Полученные навыки архитектурного мышления, умение разбивать задачу на независимые слои и обосновывать принятые технические решения представляют значительную практическую ценность для дальнейшей профессиональной деятельности.